

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
**SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY**



Project 1

Topic 10 TCP File Transferring System

Instructor : TONG VAN VAN

Student: NGUYEN NGHIA HA 20225570

 PHAM DANG TAN DUNG 20225569

ACKNOWLEDGMENT

Firstly, I would like to extend my heartfelt gratitude to my supervisor, Mr Tong Van Van, for his scientific guidance, reinforcement of my knowledge, and enthusiastic support for our project.

I would also like to sincerely thank the lecturers and students at the School of Informatics and Technology for providing the most favorable conditions for learning and research during our project.

Finally, we want to express our heartfelt appreciation to the Bach Khoa Cyber Security Center. The teachers and seniors at the center provided a deployment environment and supported us throughout the project's implementation. Thank you sincerely.

ABSTRACT

Our project focuses on developing a TCP File Transferring System using Java to address the need for secure, efficient, and user-friendly file operations in today's digital landscape. The demand for applications that allow users to manage their files, including uploading and downloading, is growing rapidly. Existing file management solutions often face challenges such as security vulnerabilities, performance issues, and complexity in user interfaces. Common approaches to solving these problems include the use of advanced encryption algorithms, optimized file handling techniques, and intuitive interface designs. However, these solutions can be limited by high development costs, increased system complexity, and potential performance bottlenecks.

The approach chosen by our team involves developing a comprehensive file management system using Java, which strikes a balance between security, performance, and ease of use. Java is selected for its platform independence, robust security features, and extensive libraries that support file handling and encryption. By leveraging these features, we aim to create an application that ensures secure file operations while providing a seamless user experience.

The solution proposed by us includes the development of a client application that offers functionalities for uploading and downloading files, as well as for encrypting and decrypting files to enhance security. The application ensures that users can securely manage their files with minimal effort, thanks to an intuitive user interface and strong encryption methods. The server side of the application is designed to handle file storage, user authentication, and secure data transactions efficiently.

The main contributions of this project include the design and implementation of a secure, efficient, and user-friendly file management application. Additionally, we develop a robust server architecture capable of managing multiple concurrent users and file transactions securely. The final outcome is a fully functional file transferring system that addresses common issues faced by existing solutions, providing users with a reliable and secure platform for managing their files.

Table of Contents

1	Introduction	5
1	Motivation	5
2	Objective and Scope of the Project	6
3	Member Contribution	7
2	Technological Methodology	8
1	SQLite	8
2	JavaFX	9
3	AES (Advanced Encryption Standard) with GCM (Galois/Counter Mode) and NoPadding	10
3	System analysis and design	10
1	Functional overview.....	10
2	Class Diagram.....	11
2.1	Client	11
2.2	Server	15
2.3	Shared	18
3	Software Quality Assurance.....	20
3.1	Definition.....	20
3.2	Code Quality Pipeline.....	21
3.3	SonarQube Analysis Results	22
3.4	Example of Code Quality Issue (Code Smell).....	23
3.5	Conclusion.....	24

4	Tools and Libraries.....	25
5	Installation and Demo.....	26
6	Conclusion.....	30
7	References.....	31

Chapter 1

Introduction

1.1 Motivation

In today's digital age, the management of personal and professional files has become a critical aspect of daily life. With the increasing volume of digital data, users require efficient and secure methods to store, access, and manage their files. Traditional file management solutions often fall short in terms of security, usability, and performance, leaving users vulnerable to data breaches, inefficient file handling, and cumbersome interfaces.

The motivation for developing this TCP File Transferring System stems from the growing need for a robust, secure, and user-friendly solution that can seamlessly handle various file operations. The application aims to provide users with a comprehensive platform where they can easily upload, download, encrypt, and decrypt their files, ensuring both convenience and security.

One of the primary motivations is to address the significant security challenges associated with file management. Data breaches and unauthorized access to sensitive files are common issues in existing solutions. By incorporating advanced encryption techniques, this application aims to provide a higher level of security, protecting users' data from potential threats.

Another motivation is the need for performance and efficiency in file operations. Users often experience slow file transfers and inefficient file handling with current applications. This project seeks to optimize these processes, providing a smoother and faster user experience.

Moreover, the project aims to simplify the user interface, making it accessible to a wide range of users, from tech-savvy individuals to those less familiar with technology.

An intuitive and user-friendly interface ensures that users can manage their files without facing unnecessary complexities.

In summary, the motivation for this TCP File Transferring System is driven by the need to enhance security, performance, and usability in file management solutions. By addressing these key areas, the project aims to provide a reliable and effective tool for users to manage their digital files securely and efficiently.

1.2 Objective and Scope of the Project

The primary objective of this project is to develop a comprehensive TCP File Transferring System using Java, designed to address the critical challenges of security, performance, and usability in file management. Key objectives include implementing advanced encryption methods to ensure the secure storage, transfer, and management of files, thus protecting user data from unauthorized access and breaches. Performance optimization is another goal, aiming to provide efficient and fast file operations to minimize delays and enhance the user experience. Additionally, the project focuses on designing an intuitive and user-friendly interface to simplify file management tasks, making the application accessible to users with varying levels of technical proficiency. Reliability is also a priority, with the development of a robust server architecture capable of handling multiple concurrent users and secure data transactions, ensuring consistent and dependable performance.

The scope of this project encompasses several essential components. Firstly, it includes the implementation of a secure login and registration system, incorporating password hashing with bcrypt for user authentication. The application provides functionalities for uploading and downloading files, with an option for users to encrypt and decrypt their files during the upload process to enhance security. An intuitive and accessible user interface is developed to allow users to manage their files with minimal effort. The server-side management involves designing and implementing an architecture that efficiently handles file storage, user authentication, and secure data transactions. The project also ensures platform

independence by leveraging Java's capabilities, allowing the application to run on various operating systems.

However, the project has certain limitations. It does not include mobile application development, focusing only on desktop applications. Advanced user roles and permissions are not part of this project's scope, as it concentrates on basic user authentication without complex role-based access controls. Additionally, extensive data analytics and reporting features are not included; the project aims to provide basic file management functionalities without in-depth analytics. By defining these objectives and scope, the project aims to deliver a secure, efficient, and user-friendly file management solution that addresses common issues faced by existing solutions, providing users with a reliable platform for managing their digital files.

1.3 Member Contribution

Student ID	Student name	Tasks Assigned
20225570	Nguyễn Nghĩa Hà	Code the Login logic Code the Register logic Use case diagram Class diagram Contribute to the project report Make Slide
20225569	Phạm Đăng Tấn Dũng	Code the main application logic GUI design Testing code using SonarQube Merging Client and Server Code the en/decrypt function Contribute to the project report

Chapter 2

Technological Methodology

In this chapter, we provide an overview of the technologies and platforms used in the development of our TCP file transferring system. For each technology or platform discussed, we will break down how it tackles a specific issue or need, we'll also explore other options and explain why we picked the one we did.

2.1 SQLite

This is a lightweight, open-source relational database management system. It is especially well-suited for applications that require simplicity, minimal setup, and low resource usage. In our case, we consider SQLite for scenarios where a full-scale database server is unnecessary, such as local testing or mobile environments. While other systems like MySQL, PostgreSQL, and MongoDB offer more advanced features for larger applications, SQLite stands out for its ease of use, zero-configuration nature, and seamless integration with small-scale applications.



Figure 2. 1: SQLite Logo

2.2 JavaFX

JavaFX is ideal for creating visually appealing, rich user interfaces for desktop applications. It offers features like animations, multimedia support, and CSS styling, which can be lacking in older frameworks like Swing. And when it comes to our UI de- signing needs, performance requirements, JavaFX is the best choice for creating visually engaging and interactive applications.



Figure 2. 2 JavaFX Logo

2.3. AES (Advanced Encryption Standard) with GCM (Galois/Counter Mode) and NoPadding

AES is a symmetric encryption algorithms, which means the same key is used for both encryption and decryption. And GCM is an authenticated encryption mode that combines the Galois field multiplication and counter mode. Because in GCM mode, padding is typically handled internally, so explicit padding is not needed. When combine two of them, the encryption ensure both data confidentiality and integrity for the en/decrypt function of the TCP-based File Transferring system

Chapter 3

System analysis and design

3.1 Functional overview

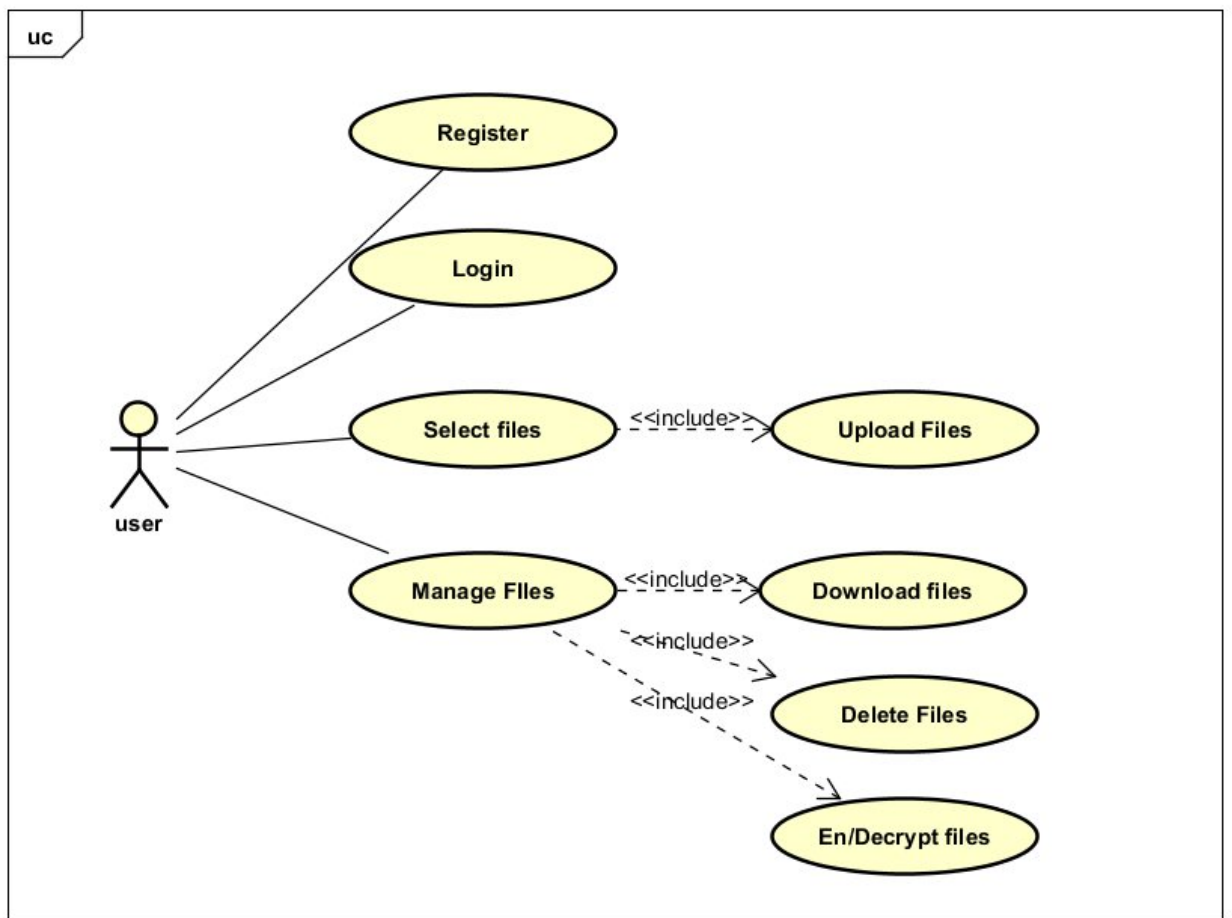


Figure 3. 1: Usecase Diagram

The above Use Case diagram illustrates the main functionalities that a user can

perform in the TCP File Transfer system. The user can register, log in, select files to upload, and manage files stored on the server. The "Manage Files" use case includes actions such as downloading, deleting, and encrypting/decrypting files. The use of <<include>> relationships indicates that functionalities like upload, download, delete, and encryption are essential parts of the file selection or management processes. This diagram provides a clear and intuitive overview of how the user interacts with the system.

3.2 Class Diagram

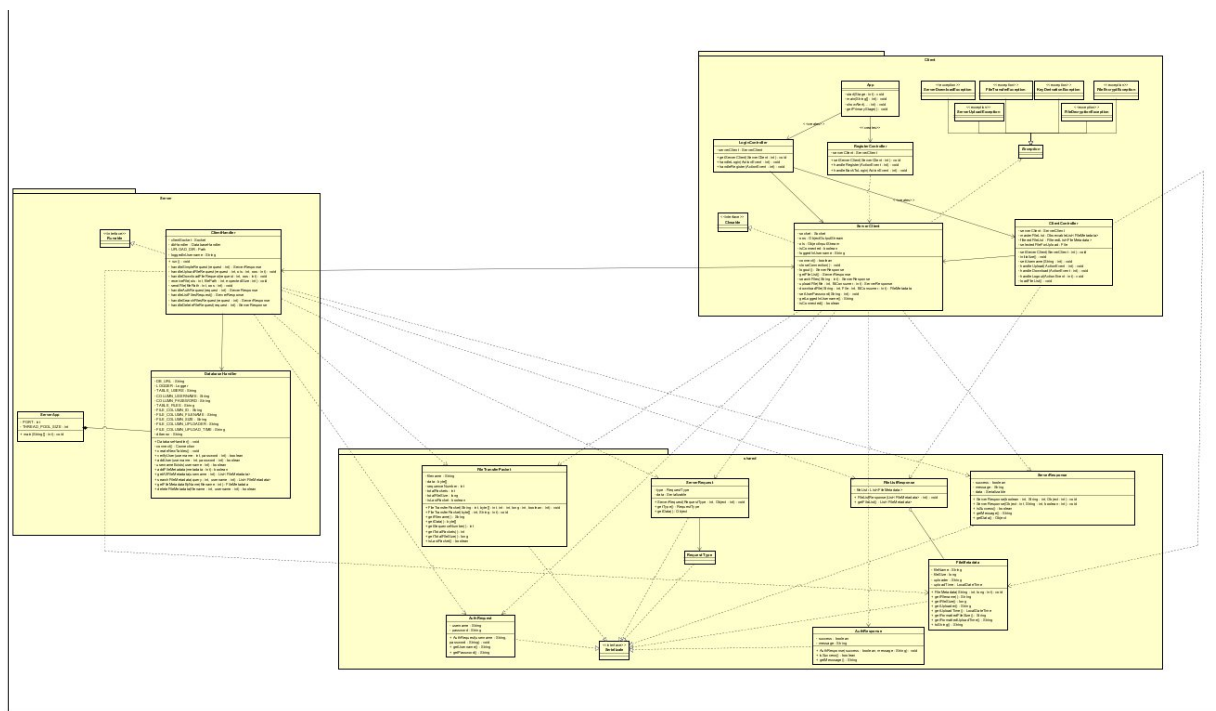


Figure 3. 2 classs diagram

3.2.1 Client

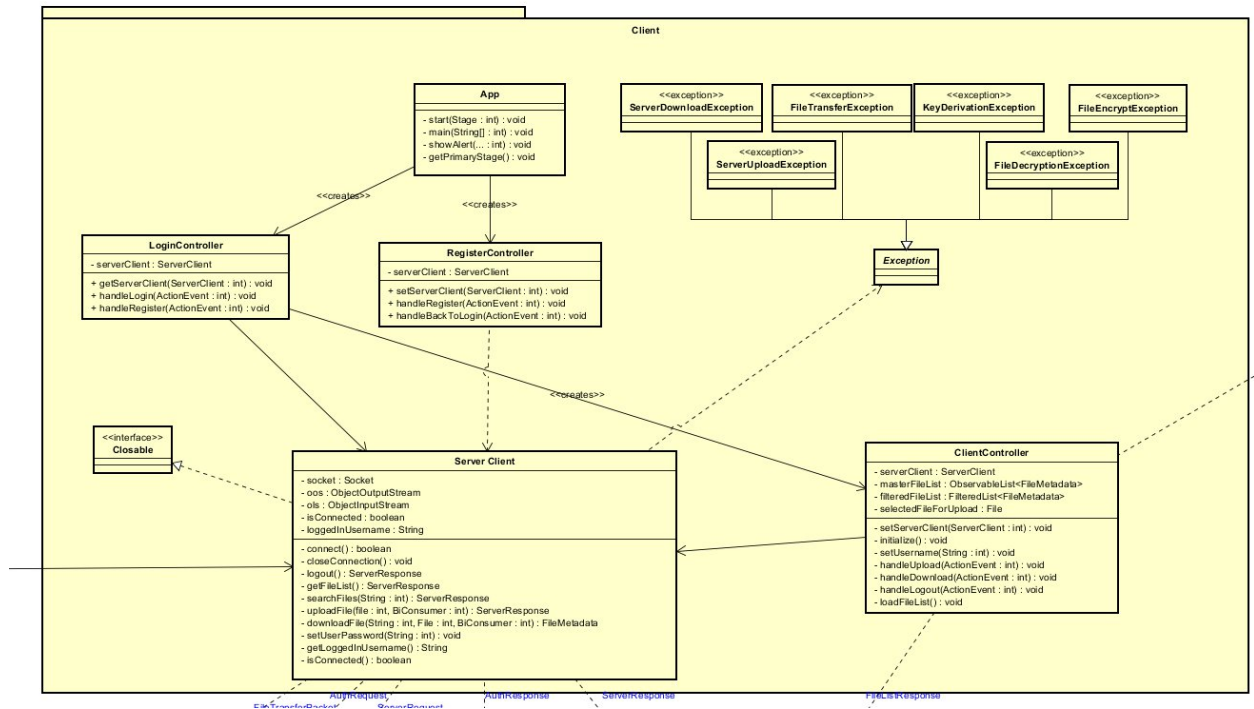


Figure 3. 3 client

The Client module of the TCP File Transfer system is a JavaFX-based user interface application that allows users to register, log in, upload, download, search for, and delete files on the server via the TCP protocol. The client communicates with the server using shared data objects, encrypts files before sending them, and displays the progress of operations to the user.

1. App

- **Description:** The main class responsible for launching the JavaFX application. It handles UI initialization and navigation between screens.
- **Attributes:** *None*
- **Methods:**
 - `start(Stage: int):` Initializes and displays the main stage.
 - `main(String[]: int):` Entry point of the application.
 - `showAlert(...): void:` Displays an alert dialog.
 - `getPrimaryStage(): void:` Retrieves the main application stage.

2. LoginController

- **Description:** Controller for the login interface. Handles user login events and transitions to the registration screen.
- **Attributes:**
 - serverClient: The client-side object that connects to the server for login operations.
- **Methods:**
 - setServerClient(ServerClient: int): Assigns a ServerClient instance.
 - handleLogin(ActionEvent: int): Handles the login action.
 - handleRegister(ActionEvent: int): Navigates to the registration screen.

3. RegisterController

- **Description:** Controller for the registration interface. Manages user registration and returning to the login screen.
- **Attributes:**
 - serverClient: Client connection used for registration actions.
- **Methods:**
 - setServerClient(ServerClient: int): Sets the ServerClient instance.
 - handleRegister(ActionEvent: int): Processes user registration.
 - handleBackToLogin(ActionEvent: int): Navigates back to the login interface.

4. ClientController

- **Description:** Main controller used after user login. Manages file uploading, downloading, searching, and logout operations.
- **Attributes:**
 - serverClient: Manages communication with the server.
 - masterFileList: The full list of files.
 - filteredFileList: Filtered list based on search.
 - selectedFileForUpload: The file selected by the user for uploading.
- **Methods:**

- setServerClient(ServerClient: int): Sets the server client instance.
- setUsername(String: int): Sets the logged-in username.
- initialize(): void: Initializes the controller and UI.
- handleUpload(ActionEvent: int): Handles file upload action.
- handleDownload(ActionEvent: int): Handles file download action.
- handleLogout(ActionEvent: int): Handles user logout.
- loadFileList(): void: Loads the list of files from the server.

5. ServerClient

- **Description:** Handles all communication between the client and the server. Responsible for login, registration, file transfer, and session management.
- **Attributes:**
 - socket: The network socket used to connect to the server.
 - oos: Output stream to send data.
 - ois: Input stream to receive data.
 - isConnected: Boolean flag to track connection status.
 - loggedInUsername: Stores the username of the current session.
- **Methods:**
 - connect(): boolean: Establishes connection to the server.
 - closeConnection(): void: Closes the server connection.
 - logout(): ServerResponse: Logs the user out.
 - getFileList(): ServerResponse: Retrieves the list of files.
 - searchFiles(String): ServerResponse: Searches files by keyword.
 - uploadFile(File, BiConsumer): void: Uploads a file with a callback.
 - downloadFile(String, File, int, BiConsumer): FileMetadata: Downloads a file from the server.
 - setUserPassword(String): void: Sets user password.
 - getLoggedInUsername(): String: Returns the current username.
 - isConnected(): boolean: Checks if the client is connected.

6. FileTransferException

Description: A custom exception used to handle errors during file transfers.

3.2.2 Server

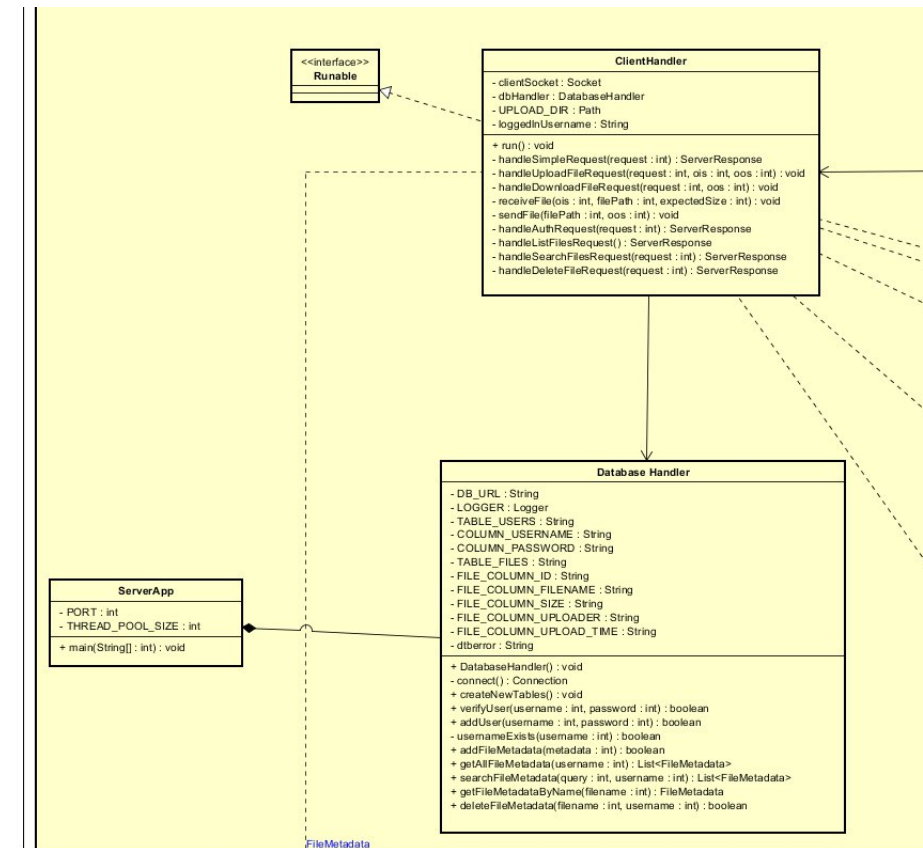


Figure 3. 4 server

The Server module of the TCP File Transfer system is responsible for listening to client connections, handling login/registration authentication, managing user and file information through an SQLite database, and processing file upload, download, search, and deletion. It communicates with the client using shared data objects over the TCP protocol. The server ensures secure file storage and provides status responses to the client for each operation.

1. ServerApp

- **Description:** Entry point of the server-side application. Responsible for initializing the server and handling multiple client connections using a thread pool.
- **Attributes:**
 - PORT: int – The port number the server listens on.
 - THREAD_POOL_SIZE: int – The number of threads available to handle clients concurrently.
- **Methods:**

- `main(String[]: int): void` – Starts the server, listens for connections, and creates `ClientHandler` instances for each client.

2. ClientHandler

- **Description:** Handles communication with a connected client. Manages file upload, download, authentication, file queries, and interaction with the database.
- **Attributes:**
 - `clientSocket: Socket` – The socket representing the client connection.
 - `dbHandler: DatabaseHandler` – Manages database operations.
 - `UPLOAD_DIR: Path` – Directory path where uploaded files are stored.
 - `loggedInUsername: String` – The username of the logged-in client.
- **Methods:**
 - `run(): void` – The main thread logic for handling client communication.
 - `handleSimpleRequest(request: int): ServerResponse` – Handles generic/simple requests.
 - `handleUploadFileRequest(request: int, ois: ObjectInputStream, oos: ObjectOutputStream, size: int): void` – Receives a file from the client.
 - `handleDownloadFileRequest(request: int, oos: ObjectOutputStream): void` – Sends a file to the client.
 - `receiveFile(...): void` – Helper method to receive file data from the client.
 - `sendFile(...): void` – Helper method to send file data to the client.
 - `handleAuthRequest(request: int): ServerResponse` – Handles user login/registration authentication.
 - `handleListFilesRequest(): ServerResponse` – Returns a list of available files.
 - `handleSearchFilesRequest(request: int): ServerResponse` – Handles file search based on a query.
 - `handleDeleteFileRequest(request: int): ServerResponse` – Handles deletion of a file.

3. DatabaseHandler

- **Description:** Manages all interactions with the underlying database, including user verification, file metadata operations, and table creation.
- **Attributes:**
 - `DB_URL: String` – Database connection URL.
 - `LOGGER: Logger` – Logger instance for logging activities.
 - `TABLE_USERS: String` – Name of the users table.

- COLUMN_USERNAME, COLUMN_PASSWORD: String – Column names for user credentials.
- TABLE_FILES: String – Name of the files table.
- FILE_COLUMN_ID, FILE_COLUMN_FILENAME, FILE_COLUMN_SIZE, FILE_COLUMN_UPLOADER, FILE_COLUMN_UPLOAD_TIME: String – Columns storing file information.
- dbError: String – Stores error message from the last DB operation (if any).
- **Methods:**
 - DatabaseHandler(): void – Constructor to initialize the handler.
 - connect(): Connection – Establishes connection with the database.
 - createNewTables(): void – Creates required tables (users and files).
 - verifyUser(username: String, password: int): boolean – Verifies user login credentials.
 - addUser(username: String, password: int): boolean – Adds a new user to the database.
 - usernameExists(username: int): boolean – Checks if a username already exists.
 - addFileMetadata(metadata: FileMetadata): boolean – Adds metadata of an uploaded file.
 - getAllFileMetadata(username: int): List<FileMetadata> – Gets all file metadata uploaded by a user.
 - searchFileMetadata(query: int, username: int): List<FileMetadata> – Searches file metadata based on query.
 - getFileMetadataByName(filename: int): FileMetadata – Gets metadata of a specific file.
 - deleteFileMetadata(filename: int, username: int): boolean – Deletes file metadata for a user.

3.2.3 Shared

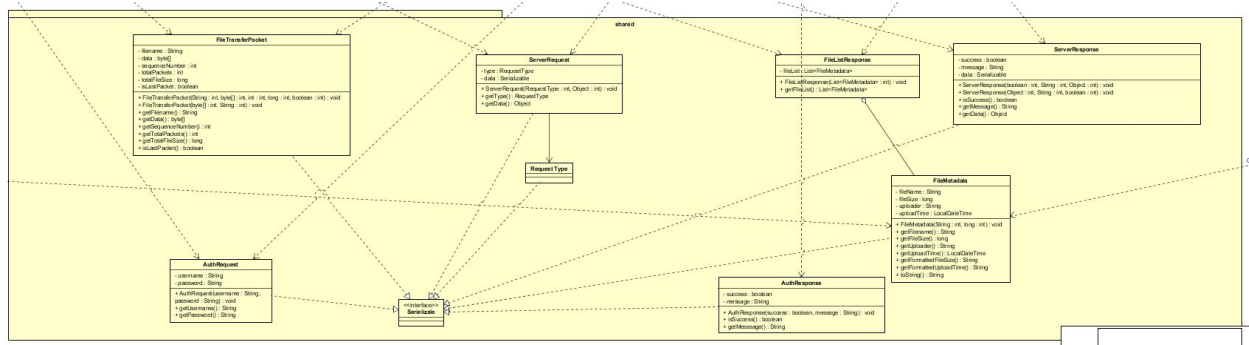


Figure 3. 5 shared folder

The Shared module contains data classes used by both the client and server, such as file information (FileMetadata), file transfer packets (FileTransferPacket), request/response objects (ServerRequest, ServerResponse), file lists (FileListResponse), and request types (RequestType). This module enables consistent, extensible, and maintainable data exchange between the client and server over TCP.

1. FileTransferPacket

- **Description:** Represents a packet used in the file transfer process. A file is split into packets for transmission.
- **Attributes:**
 - filename: String – Name of the file being transferred.
 - data: byte[] – Chunk of file data.
 - sequenceNumber: int – Order number of the packet in the sequence.
 - totalPackets: int – Total number of packets in the complete file.
 - totalFileSize: long – Total size of the file in bytes.
 - isLastPacket: boolean – Indicates whether this is the final packet.
- **Methods:**
 - FileTransferPacket(...) – Constructors to initialize the packet.
 - getFilename(): String – Returns the file name.
 - getData(): byte[] – Returns the data chunk.
 - getSequenceNumber(): int – Returns the packet's order in the sequence.
 - getTotalPackets(): int – Returns total packet count.
 - getTotalFileSize(): long – Returns the complete file size.
 - isLastPacket(): boolean – Checks if this is the last packet.

2. ServerRequest

- **Description:** Represents a request sent from the client to the server.
- **Attributes:**
 - type: RequestType – Enum indicating the type of request (e.g., upload, download, list).
 - data: Serializable – Attached request data.
- **Methods:**
 - ServerRequest(...) – Constructor for request initialization.
 - getType(): RequestType – Returns the type of request.
 - getData(): Object – Returns the request data.

3. FileListResponse

- **Description:** A specific type of server response containing a list of file metadata.
- **Attributes:**
 - fileList: List<FileMetadata> – The list of metadata for all files.
- **Methods:**
 - FileListResponse(List<FileMetadata>) – Constructor to initialize with a file list.
 - getFileList(): List<FileMetadata> – Returns the list of file metadata.

4. FileMetadata

- **Description:** Contains metadata about a file, such as its name, size, uploader, and upload time.
- **Attributes:**
 - fileName: String – The name of the file.
 - fileSize: long – Size of the file in bytes.
 - uploader: String – Username of the person who uploaded the file.
 - uploadTime: LocalDateTime – Time when the file was uploaded.
- **Methods:**
 - FileMetadata(...) – Constructor to initialize file metadata.
 - getFilename(): String – Returns the file name.
 - getFileSize(): long – Returns file size.
 - getUploader(): String – Returns the uploader's name.
 - getUploadTime(): LocalDateTime – Returns the upload timestamp.
 - getFormattedFileSize(): String – Returns the size as a formatted string (e.g., "2.3 MB").

- getFormattedUploadTime(): String – Returns the upload time as a readable string.
- toString(): String – Returns a string representation of the file metadata.

5. ServerResponse

- **Description:** Represents a general response from the server to the client, indicating success, message, and optional data.
- **Attributes:**
 - success: boolean – Indicates whether the request was successful.
 - message: String – Optional message returned with the response.
 - data: Serializable – Optional data (can be metadata, file list, etc.).
- **Methods:**
 - ServerResponse(...) – Constructors to initialize the response.
 - isSuccess(): boolean – Returns whether the response indicates success.
 - getMessage(): String – Returns the message.
 - getData(): Object – Returns the attached response data.

3.3 Software Quality Assurance

3.3.1 Definition

Software Quality Assurance (SQA) is a critical component of the software development process that ensures the quality and reliability of software products. SQA encompasses methodologies and practices designed to monitor the software engineering processes and methods used to ensure quality. The emphasis is on preventing defects rather than detecting them after they have been introduced.

The main goal of software QA testing is to detect and address any defects or issues in the software product or application before its release. This thorough process includes testing for functionality, performance, security, usability, and compatibility across different platforms and devices.

Software QA testing is typically carried out by a team of testers. Their tasks

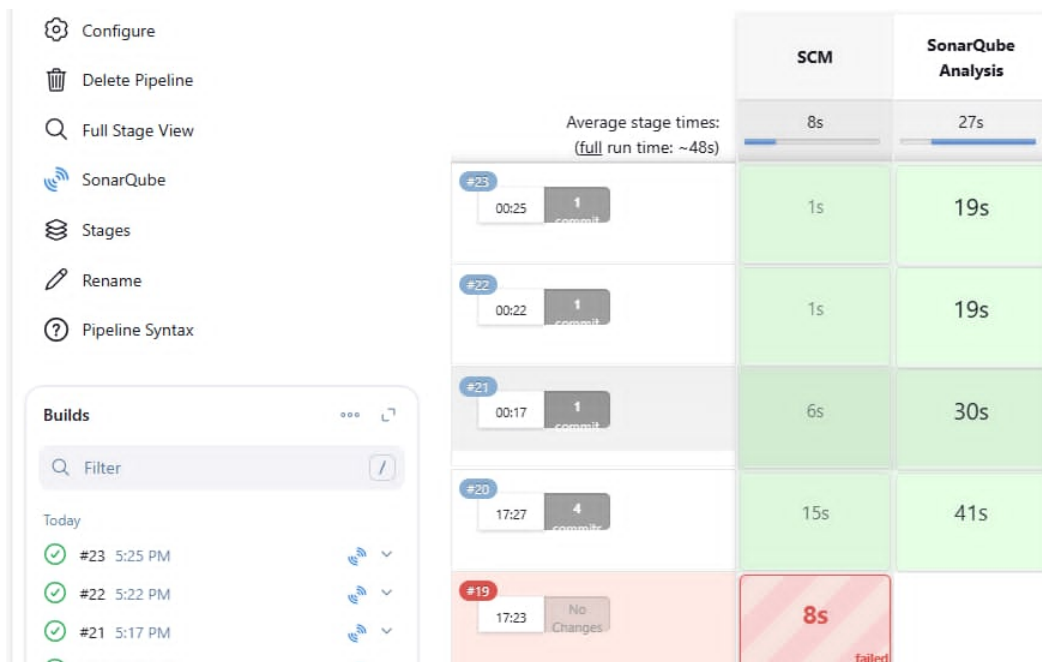
include planning, designing, executing, and reporting on tests. The testing process may incorporate automated techniques, depending on the software product's complexity. However, given the project's scope and our expertise limitations, we will primarily rely on manual testing methods, supplemented by freely available analytical tools.

The primary aim of software quality assurance (QA) testing is to deliver a high-quality software product or application that meets the needs of users and stakeholders, ensuring its reliability, security, and user-friendliness.

3.3.2 Code Quality Pipeline

The pipeline is configured with key steps: fetching source code from a version control system (SCM), then analyzing the code with SonarQube.

Image below shows the build history of the pipeline, along with execution time for each stage:



Analysis:

- Each pipeline run takes approximately 35 seconds on average.
- The SCM stage takes about 8 seconds, while the SonarQube Analysis stage

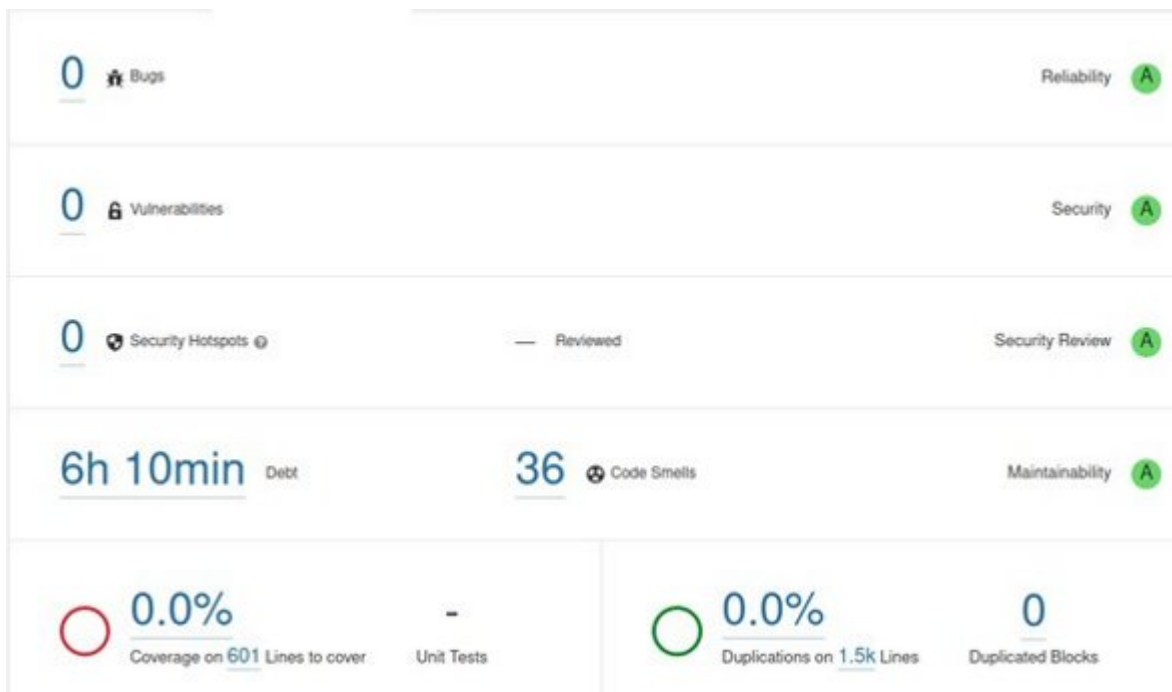
consumes the most time—about 27 seconds.

- All recent builds were successful (green), except for build #19, which failed at the SCM stage due to an unspecified error.

This demonstrates that the quality control process is well-integrated and functioning stably, providing timely support to developer.

3.3.3 SonarQube Analysis Results

After the code is analyzed, SonarQube provides detailed reports on quality metrics such as reliability, security, maintainability, and code smells.



Analysis:

- No bugs, vulnerabilities, or security hotspots were found, indicating a high level of stability and security.
- However, 36 code smells were detected, with an estimated technical debt of 6 hours and 10 minutes, affecting maintainability. Though, we have fixed all of those.
- All metrics received an A rating – the highest quality grade.

This is a highly positive result, though further efforts should be made to reduce code

smells and improve maintainability.

3.3.4 Example of Code Quality Issue (Code Smell)

SonarQube not only provides an overview but also details each issue with explanations and suggestions for fixing them



Issue Analysis:

- Rule: java:S112 – Define and throw a dedicated exception instead of using a generic one.
- Type: Code Smell (Major) – affects code readability and maintainability.
- Status: Closed (Fixed) → this issue has been successfully resolved.

Fixing this issue enhances the clarity and reliability of the code, aligning with Clean Code principles.

3.3.5 Conclusion

Through integration and use of SonarQube:

- Continuous quality control is ensured throughout software development.
- Issues are detected early, reducing debugging and maintenance time.
- The development team responded quickly and effectively addressed major code smells.

Chapter 4

Tools and Libraries

Uses	Tools	URL
IDE	Visual studio code	https://code.visualstudio.com/
GUI framework	JavaFX	https://openjfx.io/
UI design	Scene Builder	https://gluonhq.com/products/scene-builder/
Draw usecase and class diagrams	Astah UML	https://astah.net/
Manages project de-pendencies	Maven version 3.9.9	https://maven.apache.org/
Code quality analyze	SonarQube	https://www.sonarsource.com/products/sonarqube/
Task Automation	Jenkins LTS	https://www.jenkins.io/
Manage source code version	Github	https://github.com/

Chapter 5

Installation and Demo

1.Client Demo

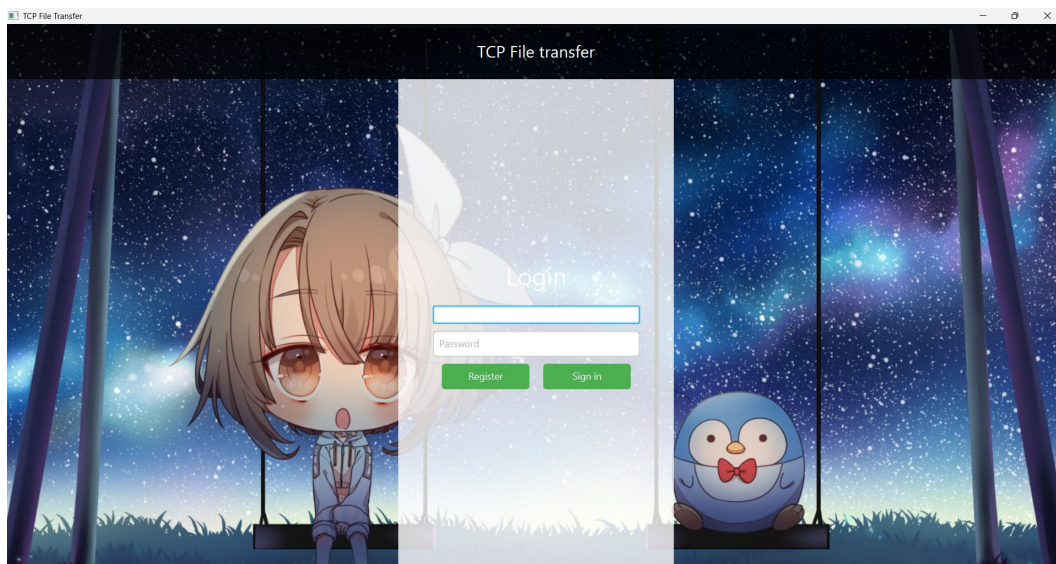


Figure 5. 1: Login view

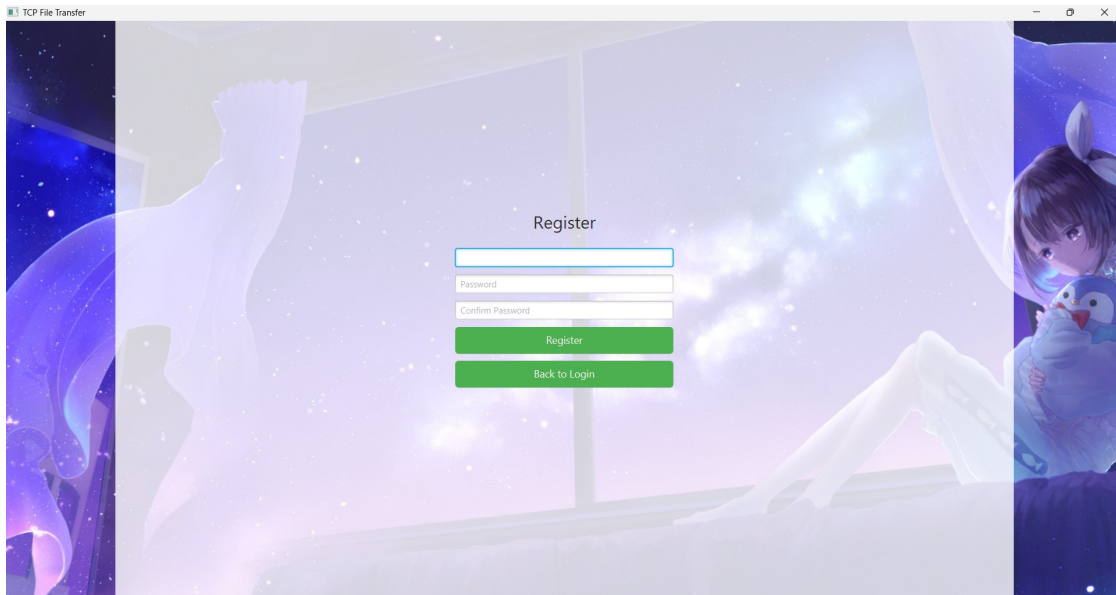


Figure 5. 2: Register view

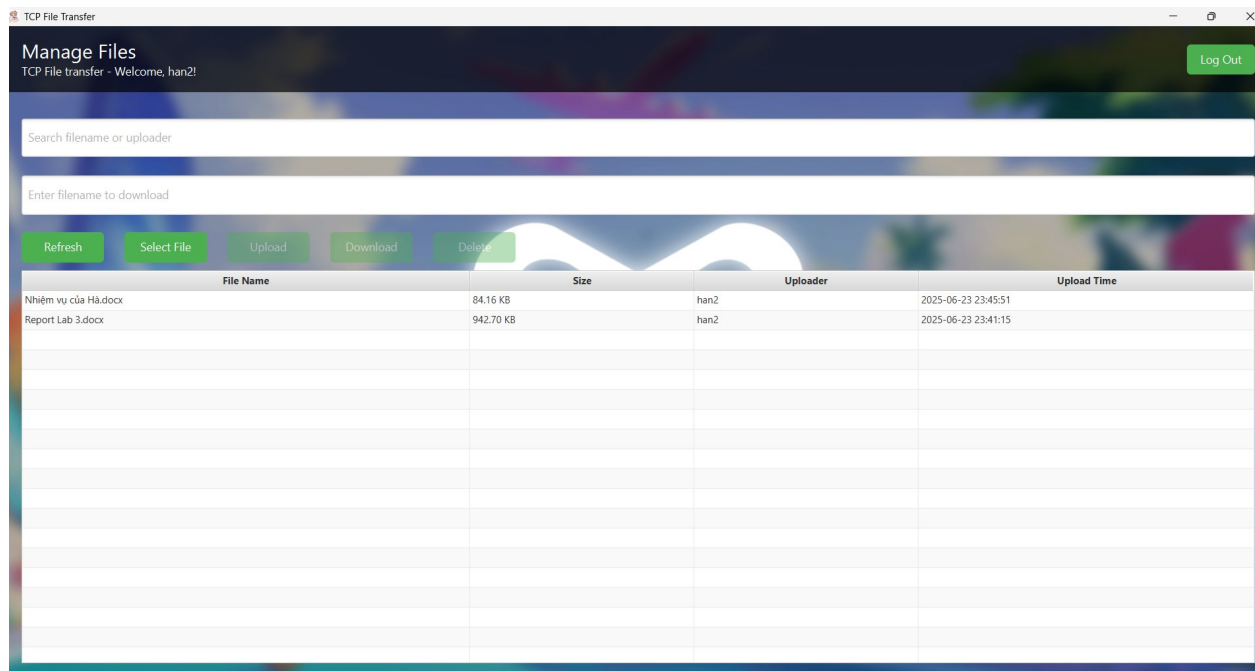


Figure 5. 3: Home screen view

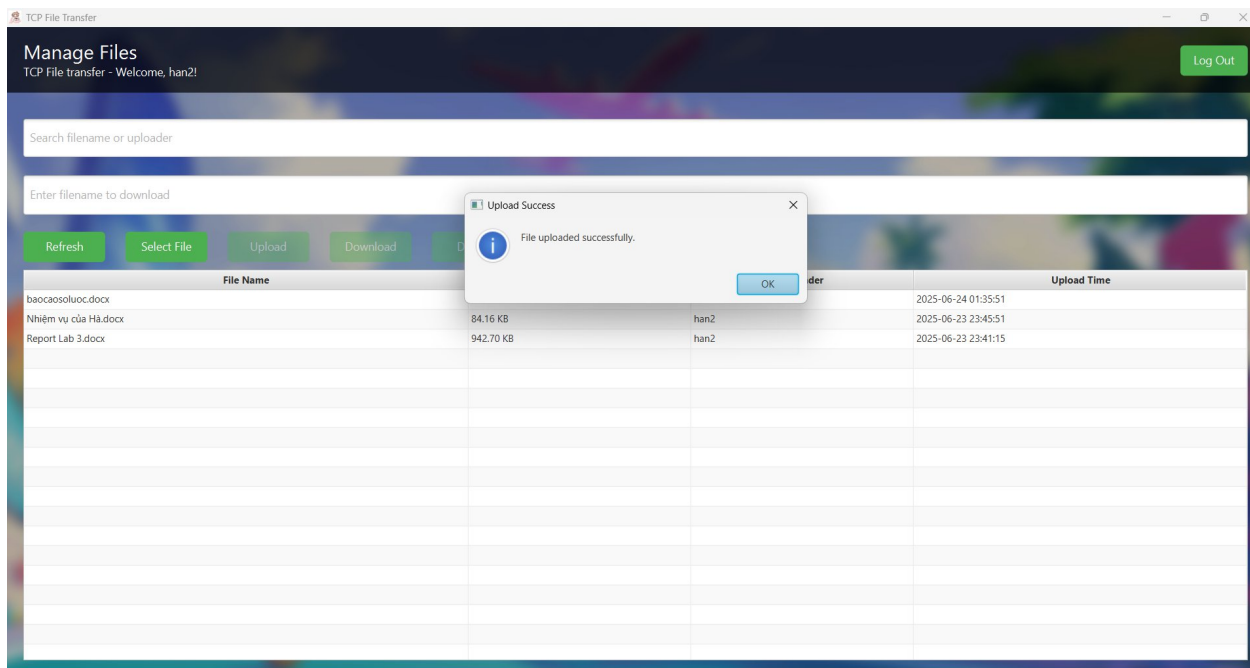


Figure 5. 4: Upload success

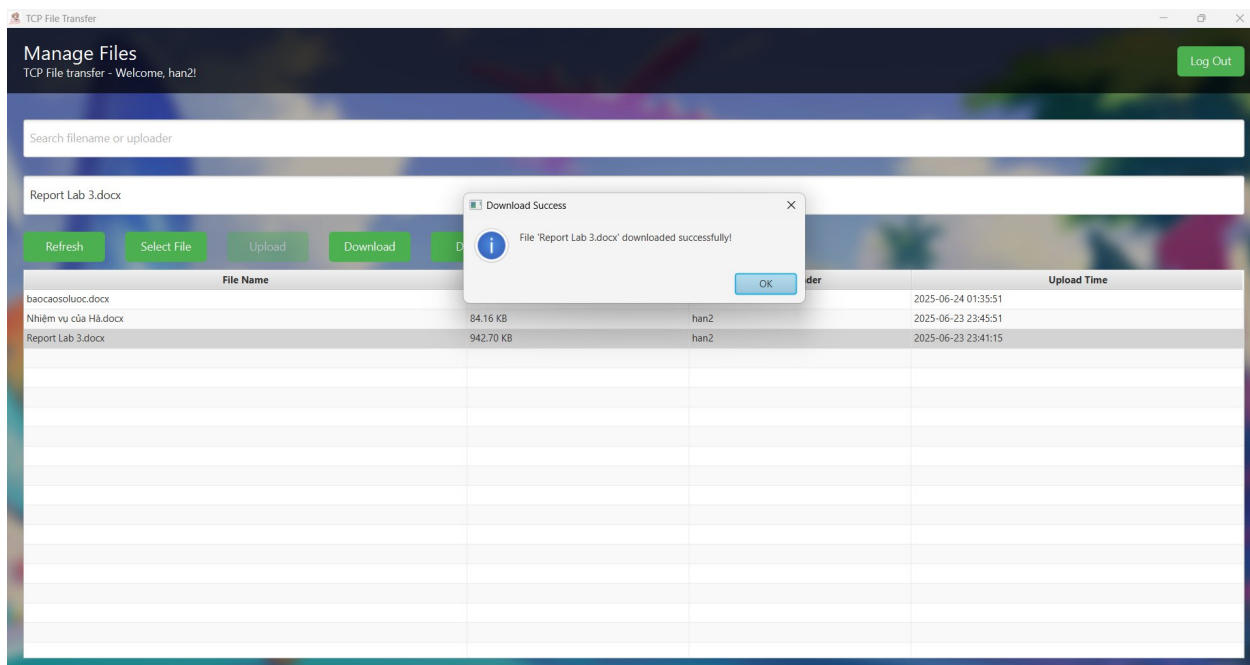


Figure 5. 5 Download success

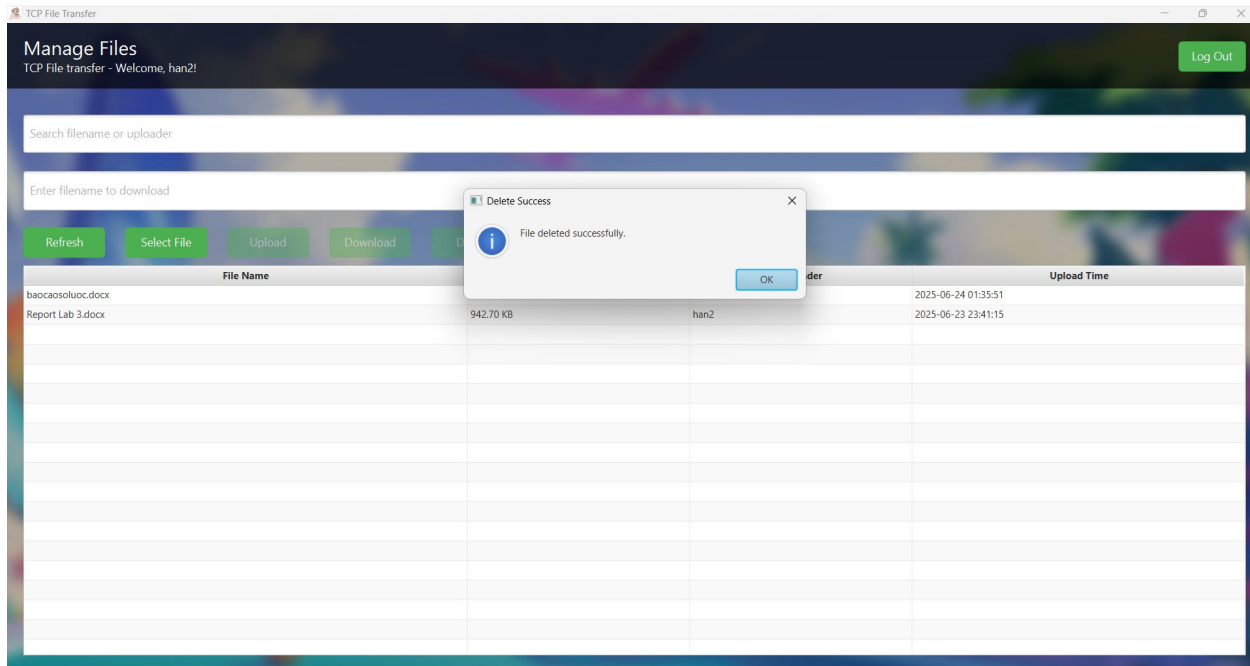


Figure 5. 6 Delete success

PRACTICE REPORT NO. 3

ANALYSIS OF ARP CACHE POISONING ATTACK TECHNIQUES

Student's full name:

Student ID:

Practice Result Requirements:

Pack the results into a compressed file named in the format **HoTenSV_MSSV_Lab03.zip**

- Report on practice results.
- The file capturing traffic (wireshark or tcpdump) on the surveillance machines for each scenario. Each file size should not exceed **5 MB**.

Container	Interface	IP Address/Netmask	MAC Address
User		10.17.70.5	02:42:0A:11:46:05
Attacker		10.17.70.105	02:42:0A:11:46:69
Server		10.17.70.6	02:42:0A:11:46:06

PRACTICE RESULTS

1. Deploy the system (2 points)

Deploy the virtualization system for the lab, take screenshots of the results, and include them in

Figure 5. 7 decrypt file

When a file is uploaded to the system, it is encrypted. When the file is downloaded, it is decrypted, allowing us to view its contents.

Chapter 6

Conclusion

The “TCP File Transfer” project has successfully developed a file transmission and management system over a network based on the client-server model using the TCP protocol. The system consists of three main components: the client (user interface), the server (business logic and storage), and the shared module (common data classes). Each component is designed independently to ensure scalability, easy maintenance, and future upgrades.

On the client side, the application provides a user-friendly interface that allows users to register, log in, upload, download, search for, and delete files with ease. Each operation is accompanied by clear status feedback, enhancing the overall user experience. The client communicates with the server through shared data objects, ensuring consistency and safety during data transmission.

The server is responsible for user authentication, managing file and user information via an SQLite database, and handling requests from the client such as file upload, download, search, and deletion. It ensures secure file storage, supports concurrent client connections, and accurately responds to operation statuses.

The shared module acts as a mediator, defining the data classes and communication protocols used between the client and server. This approach minimizes communication errors and increases flexibility when extending or modifying the system.

Experimental results show that the system operates stably, fulfills the intended functionalities, and enables fast and secure file transfers. The clear modular separation makes it easy to integrate advanced features such as user access control, data encryption, or the development of web/mobile interfaces in the future.

In conclusion, the project not only meets functional requirements but also adheres to principles of software architecture, security, and scalability. It serves as a solid foundation for building modern file transfer systems that support real-world needs in learning, work, and data sharing over networks.

Chapter 7

References

1. CD Foundation. Jenkins. <https://www.jenkins.io/>.
2. Openjfx. JavaFX. <https://openjfx.io/>.
3. GluonHQ. Scene Builder. <https://gluonhq.com/products/scene-builder/>.
4. SonarSource. Sonarqube. <https://www.sonarsource.com/products/sonarqube/>.
5. The Apache Software Foundation. Maven. <https://maven.apache.org/>.
6. Astah. <https://astah.net/products/astah-community/>.